

Shape Analysis Methods for Images

This document provides a summary of various methods used for analyzing shapes in images, along with code snippets for each method.

1. Contour-Based Features

- Contours represent the boundary of objects in an image.
- Features: Area, Perimeter, Aspect Ratio, Circularity, Eccentricity.
- Useful for detecting and analyzing basic shapes.

Code:

```
```python
```

```
import cv2
```

```
import numpy as np
```

```
def compute_contour_features(image):
```

```
 gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
```

```
 edges = cv2.Canny(gray, 100, 200)
```

```
 contours, _ = cv2.findContours(edges, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
```

```
 features = []
```

```
 for contour in contours:
```

```
 area = cv2.contourArea(contour)
```

```
 perimeter = cv2.arcLength(contour, True)
```

```
 x, y, w, h = cv2.boundingRect(contour)
```

```
 aspect_ratio = w / h
```

```
circularity = (4 * np.pi * area) / (perimeter**2 + 1e-6)
```

```
features.append([area, perimeter, aspect_ratio, circularity])
```

```
return np.mean(features, axis=0)
```

```
```
```

2. Hu Moments

- Hu Moments are shape descriptors invariant to rotation, translation, and scaling.
- Useful for comparing shapes in different orientations.

Code:

```
```python
```

```
def compute_hu_moments(image):
```

```
 gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
```

```
 _, binary = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)
```

```
 moments = cv2.moments(binary)
```

```
 hu_moments = cv2.HuMoments(moments).flatten()
```

```
 return -np.sign(hu_moments) * np.log10(np.abs(hu_moments))
```

```
```
```

3. Fourier Descriptors

- Represents the contour in the frequency domain.
- Useful for comparing shapes invariant to transformations like rotation and scale.

Code:

```

```python
def compute_fourier_descriptors(image, num_coefficients=10):

 gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

 edges = cv2.Canny(gray, 100, 200)

 contours, _ = cv2.findContours(edges, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

 if contours:

 contour = max(contours, key=cv2.contourArea).squeeze()

 complex_contour = contour[:, 0] + 1j * contour[:, 1]

 fourier_descriptors = np.fft.fft(complex_contour)

 return np.abs(fourier_descriptors[:num_coefficients])

 return None
```

```

4. Skeletonization (Medial Axis Transformation)

- Reduces the shape to its centerline (skeleton), preserving topology.
- Useful for analyzing shapes and topological features.

Code:

```

```python
from skimage.morphology import skeletonize

from skimage.measure import regionprops

def compute_skeleton_features(image):

 gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

 _, binary = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)

```

```

skeleton = skeletonize(binary // 255)

properties = regionprops(skeleton.astype(int))

if properties:

 length = properties[0].perimeter

 return np.array([length])

return None
'''

```

## 5. Shape Matching (Hausdorff or Chamfer Distance)

- Measures similarity between two shapes by comparing their contours.
- Hausdorff Distance is used for maximum point-to-point comparison.

Code:

```

'''python

def compute_hausdorff_distance(image1, image2):

 gray1 = cv2.cvtColor(image1, cv2.COLOR_BGR2GRAY)

 gray2 = cv2.cvtColor(image2, cv2.COLOR_BGR2GRAY)

 edges1 = cv2.Canny(gray1, 100, 200)

 edges2 = cv2.Canny(gray2, 100, 200)

 contours1, _ = cv2.findContours(edges1, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)

 contours2, _ = cv2.findContours(edges2, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)

```

if contours1 and contours2:

    contour1 = max(contours1, key=cv2.contourArea).squeeze()

    contour2 = max(contours2, key=cv2.contourArea).squeeze()

    distance = cv2.createHausdorffDistanceExtractor().computeDistance(contour1, contour2)

    return distance

return None

```

6. Bounding Box Features

- Computes features from the bounding box of an object.
- Features: Aspect Ratio, Extent, Solidity.

Code:

```python

def compute\_bounding\_box\_features(image):

    gray = cv2.cvtColor(image, cv2.COLOR\_BGR2GRAY)

    edges = cv2.Canny(gray, 100, 200)

    contours, \_ = cv2.findContours(edges, cv2.RETR\_EXTERNAL, cv2.CHAIN\_APPROX\_SIMPLE)

    if contours:

        contour = max(contours, key=cv2.contourArea)

        x, y, w, h = cv2.boundingRect(contour)

        aspect\_ratio = w / h

        area = cv2.contourArea(contour)

        extent = area / (w \* h)

        hull\_area = cv2.contourArea(cv2.convexHull(contour))

```
solidity = area / hull_area
```

```
return np.array([aspect_ratio, extent, solidity])
```

```
return None
```

```
'''
```